Some Notes on Computability

The Turing machine represents the culmination of a series of increasingly powerful abstract computing devices. The Church-Turing thesis, proposed by logician Alonzo Church in 1936, asserts that any effective computation in any algorithmic system can be accomplished using a Turing machine. The formulation of this thesis for decision problems and its implication for computability are discussed below.

A decision problem consists of a set of questions whose answers are either **yes** or **no**. A solution to a decision problem is an effective procedure that determines the answer for each question in the set. A decision problem is undecidable if there is no algorithm that solves the problem. The ability of Turing machines to return affirmative and negative responses makes them an appropriate mathematical system for constructing solutions to decision problems. The Church-Turing thesis asserts that a Turing machine can be designed to solve any decision problem that is solvable by any effective procedure. Consequently, to establish that a problem is unsolvable it suffices to show that there is no Turing machine solution.

A solution to a decision problem **P** is an algorithm that determines the appropriate answer to every question $\mathbf{p} \in \mathbf{P}$.

Since a solution to a decision problem is an algorithm, a review of our intuitive notion of algorithmic computation may be beneficial. We have not defined—and probably cannot precisely define—*algorithm*. This notion falls into the category of "I can't describe it but I know one when I see one". We can, however, list several properties that seem fundamental to the concept of algorithm. An algorithm that solves a decision problem should be:

- Complete: It produces an answer, either positive or negative, to each question in the problem domain.

- Mechanistic: It consists of a finite sequence of instructions, each of which can be carried out without requiring insight, ingenuity, or guesswork.

- Deterministic: When presented with identical input, it always produces the same result.

A procedure that satisfies the preceding properties is often called *effective*.

The computations of a standard Turing machine are clearly mechanistic and deterministic. A Turing machine that halts for every input string is also complete.

The Church-Turing thesis asserts that every solvable problem can be transformed into an equivalent Turing machine problem.

The Church-Turing thesis for decision problems: There is an effective procedure to solve a decision problem if, and only if, there is a Turing machine that halts for all input strings and solves the problem.

The Church-Turing thesis is not a mathematical theorem; it cannot be proved. This would require a formal definition of the intuitive notion of an effective procedure. The claim could, however, be disproved. This could be accomplished by discovering an effective procedure that cannot be computed by a Turing machine. There is an impressive pool of evidence that suggests that such a procedure will not be found. Perhaps the strongest evidence supporting Church-Turing thesis is that all known effective procedures have been able to be transformed into equivalent Turing machines.

The Church-Turing thesis may be considered to provide a definition of algorithmic computation. This is a very limiting viewpoint. There are many systems that satisfy our intuitive notion of an effective algorithm. For example, the machines designed by Post [1936], recursive functions defined by Kleene [1936], the lambda calculus of Church [1941], and more recently, programming languages for digital computers.

Perhaps the strongest evidence supporting Church-Turing thesis is that all known effective procedures have been able to be transformed into equivalent Turing machines.

The Church-Turing thesis asserts that a decision problem **P** has a solution if, and only if, there is a Turing machine that determines the answer for every $\mathbf{p} \in \mathbf{P}$. If no such Turing machine exists, the problem is said to be **undecidable** (unsolvable).

The Halting Problem for Turing Machine

The most famous of the undecidable problems is concerned with the properties of Turing machines themselves. The halting problem may be formulated as follows: Given an arbitrary Turing machine M with input alphabet Σ and a string $w \in \Sigma^*$, will the computation of M with input w halt? It can be shown that there is no algorithm that solves the halting problem. For a proof see the separate handout.

A List Of Some Undecidable Problems

The Halting Problem (V 2): Is there an algorithm that can decide whether the execution of an arbitrary program halts on an arbitrary input?

The Halting Problem (V 3): Is there a computable function that can decide whether the execution of an arbitrary computable function halts on an arbitrary input?

The Total Problem: Is there an algorithm that tells whether an arbitrary computable function is total?

The Equivalence Problem: Does there exist an algorithm that can decide whether two arbitrary computable functions produce the same output?

The Busy beaver Problem: For an arbitrary value of n, can $\Sigma(n)$ be computed?